

實驗作品名稱:音樂聖誕卡片

實驗目的&內容:

1. 實驗目的:本實驗目的在於於8051微控制器實習板上,透過板上之連接埠、按鍵、蜂鳴器、LCD文字顯示器以及8*8LED點陣顯示器,完成正確之硬體接線配置後,於電腦端的SIMLAB8051中撰寫對應之C語言控制程式,並將程式燒入至8051微控制器中執行,程式時做出的內容包含"多模式切換控制、音樂播放、8*8點陣圖形的顯示、LCD跑馬燈文字顯示,等功能",並利用Timer中斷產生蜂鳴器音階與節奏控制,並同步進行LED點陣顯示動畫與LCD文字捲動顯示,使用者可透過實習板上的按鍵進行顯示模式切換,使系統於不同模式下分別呈現對應之音樂、圖形與文字顯示效果,達到多媒體整合輸出的實作目標。本實驗作品即為本學期微算機實驗課程之期末驗收成果之一。

2. 實驗內容:

- (1) 程式碼(C語言)要做到的內容方法及規則:

內容:

本程式主要結合8*8LED點陣顯示器、LCD文字顯示器、蜂鳴器與按鍵輸入,於8051微控制器上實作一套多模式音樂顯示系統,系統共分為三種顯示模式,使用者可透過按鍵進行模式切換,不同模式下會顯示對應之點陣圖案、LCD跑馬燈文字與音樂內容。

三種模式分別為:

1. 聖誕模式(MODE_XMAS):

顯示聖誕樹圖案,LCD顯示"Merry Christmas"跑馬燈文字,並播放Merry Christmas音樂。

2. 新年模式(MODE_NEWYR):

顯示表情圖案,並於有笑臉與無表情臉之間交替切換,LCD顯示"Happy New Year"跑馬燈文字,並播放快樂頌音樂。

3. 星星模式(MODE_STAR):顯示固定星星圖案,LCD顯示"Twinkle Star"跑馬燈文字,並播放Twinkle Star音樂。

並且所有模式皆透過同一顆按鍵進行循環切換,切換模式時會同步更換音樂段落、LCD顯示文字與點陣顯示內容。

詳細方法:

- 主程式架構(main與主迴圈):

程式的進入點為main(),一開始會先將P0、P1、P2、P3全部設為0xFF,確保所有I/O腳位在初始化時為高電位,避免誤觸發外部裝置,接著呼叫LCD_Init()初始化LCD顯示器,去完成顯示模式設定與清屏動作,在Timer0與Timer1部分則是都設定為Mode 1(16-bit計時模式),其中Timer0負責音高的產生,Timer1負責節奏控制與動畫更新,完成Timer設定後,會開Timer0、Timer1中斷與全域中斷,再正式進入系統執行狀態。

主程式進入while(1)無限迴圈後,會持續執行以下兩項工作:

1. 呼叫displayPattern()掃描顯示8*8LED點陣
2. 呼叫checkButton()偵測按鍵是否被按下以切換模式

- 模式切換流程(checkButton 與 switchMode):

按鍵S1接於P1.4,為低電位,checkButton()會記錄前一次按鍵狀態,利用由高轉低的方式偵測按鍵下的降沿,避免長按造成多次誤觸,然後當偵測到有效按鍵輸入時,會先加入短暫延遲作為防彈跳處理,再呼叫switchMode()進行模式切換,switchMode()則會將currentMode依序在MODE_XMAS、MODE_NEWYR、MODE_STAR間做循環,並同時完成以下初始化動作:

1. 重置音樂索引idx
2. 重置節奏計數tick與短休止狀態shortRest
3. 重置LCD跑馬燈起始位置lcdPos
4. 重置模式下的表情切換狀態

依據不同模式,idx會指向對應歌曲的起始位置,並同步更換LCD顯示的文字內容。

- 音樂播放控制(Timer0中斷):

Timer0中斷副程式負責蜂鳴器音高產生,每次Timer0中斷發生時,會依據Song[idx]的值決定輸出狀態,若目前音符為休止符(值為0)或系統正處於短休止狀態,蜂鳴器輸出腳位P1.3會被關閉,不然會透過翻轉P1.3做輸出,形成方波訊號,其頻率由Timer0的重載值決定,進而去產生對應音階的聲音。

- 節奏與動畫控制(Timer1中斷):

Timer1中斷為整個系統的節奏核心,負責多項同步控制功能,Timer1每次中斷時會:

1. 增加音樂節奏計數tick
2. 增加LCD捲動計數lcdTick
3. 更新8*8點陣閃爍節拍blinkTick
4. 在新年模式下更新表情切換計數smileTick

當tick達到Dur[idx]*2時,系統會進入短休止狀態shortRest,作為音符之間的時間,短休止結束後,idx會加一切換至下一個音符,並依目前模式限制idx在對應歌曲範圍內做循環。

- LCD顯示與跑馬燈控制:

LCD使用8-bit模式，資料線接於P0，控制線則接於P1.0~P1.2，LCD_Scroll()副程式負責是LCD跑馬燈顯示，每次Timer1中斷達到指定次數後，會重新寫入LCD第一行的16個字元，文字會透過循環索引方式讀取字串內容，使顯示文字可以不斷向左捲動，達成跑馬燈效果。

- 8×8 LED點陣顯示控制(displayPattern):

是8×8 LED點陣顯示採用逐列掃描方式，displayPattern()會依序選擇8列中的其中一列，透過P3輸出列選擇訊號，並由P2輸出該列對應的圖案資料，系統也會根據currentMode去選擇不同的圖案資料(對應的):

1. MODE_XMAS 顯示聖誕樹圖案
2. MODE_NEWYR 顯示表情圖案
3. MODE_STAR 顯示星星圖案

在非表情模式下，點陣會依blinkState進行整體閃爍；在表情模式下，點陣會切換表情，以增加顯示的動態效果。

- 整體作業之特性說明:

本作業透過Timer0與Timer1分工，將音高、節奏、顯示與動畫同步進行，主程式僅負責顯示掃描與按鍵偵測，所有時間相關控制皆由中斷處理，會確保系統在播放音樂的同時，仍能穩定顯示對應的8×8 LED點陣畫面與LCD文字列印。

(2) 連接埠和功能對應:

連接埠(功能)	連接埠(port)
8*8點陣_ROW:R1~R8	P3.0~P3.7
8*8點陣_LOW:C1~C8	P2.0~P2.7
Botton S1	P1.4
蜂鳴器	P1.3、GND
LCD RS	P1.0
LCD RW	P1.1
LCD E	P1.2
DB0~DB7	P0.0~0.7

(3) 音樂聖誕卡片code:

音樂聖誕卡片C語言(code):

一、標頭檔與型別定義區	
#include <REG51.H>	// 引入8051的硬體定義檔，裡面包含P0~P3、Timer、中斷等設定
#include <string.h>	// 引入字串函式庫，之後會用到strlen()、strcpy()
#define uchar unsigned char	// 把unsigned char簡寫成uchar，之後寫程式比較方便
#define uint unsigned int	// 把unsigned int簡寫成uint
二、硬體腳位定義區(LCD/蜂鳴器/按鍵)	
sbit RS = P1^0;	// LCD的RS腳接在P1.0，用來選擇是指令還是資料
sbit RW = P1^1;	// LCD的RW腳接在P1.1，用來選擇寫入或讀取
sbit EN = P1^2;	// LCD的EN腳接在P1.2，用來觸發LCD接收資料
sbit BZ = P1^3;	// 蜂鳴器接在P1.3，用來輸出聲音
sbit S1 = P1^4;	// 按鈕接在P1.4，按下時為低電位
三、模式定義與系統狀態變數	
#define MODE_XMAS 0	// 定義模式0為聖誕模式
#define MODE_NEWYR 1	// 定義模式1為表情(會笑)模式
#define MODE_STAR 2	// 定義模式2為星星模式
uchar currentMode = MODE_XMAS; // 用來記錄目前是哪一个模式，預設從聖誕模式開始	
四、音階定義區(Timer0音高用)	
#define C4 0xF22A	// 定義中音 Do 的Timer0重載值

```
#define D4 0xF3C2      // 定義中音 Re
#define E4 0xF536      // 定義中音 Mi
#define F4 0xF5FD      // 定義中音 Fa
#define G4 0xF729      // 定義中音 Sol
#define A4 0xF7D0      // 定義中音 La
#define B4 0xF8E4      // 定義中音 Si
#define C5 0xF914      // 定義高音 Do
#define D5 0xF9E0      // 定義高音 Re
#define E5 0xFA9A      // 定義高音 Mi
#define F5 0xFAFE      // 定義高音 Fa
```

五、8*8 LED點陣圖案資料區

```
uchar code christmasTree[8] = { // 聖誕樹的8*8點陣圖，每一個數字代表一列
    0x18,0x3C,0x7E,0x18,0x3C,0x7E,0xFF,0x18
};

uchar code smileyFace[8] = { // 笑臉圖案的8*8點陣資料
    0xFF,0x81,0xA5,0x81,0xA5,0xBD,0x81,0xFF
};

uchar code neutralFace[8] = { // 沒有笑的表情圖案
    0xFF,0x81,0xA5,0x81,0x81,0xFF,0x81,0xFF
};

uchar code star[8] = { // 星星圖案的8*8點陣資料
    0x02,0x07,0x22,0x70,0x24,0x4E,0xE4,0x40
};
```

六、音樂資料與節奏資料區

```
unsigned int code Song[] = { // 三首歌的音階資料陣列，0表示休止符
    G4,C5,C5,D5,C5,B4,A4,A4,A4, // Merry Christmas
    D5,D5,E5,D5,C5,B4,G4,G4,
    E5,E5,F5,E5,D5,C5,A4,
    G4,G4,A4,D5,B4,C5,
    0,0,0,0,0,0, // 停頓

    E4,E4,F4,G4,G4,F4,E4,D4, // 快樂頌
    C4,C4,D4,E4,E4,D4,D4,
    E4,E4,F4,G4,G4,F4,E4,D4,
    C4,C4,D4,E4,D4,C4,C4,
    0,0,0,0,0,0, // 停頓

    C4,C4,G4,G4,A4,A4,G4, // Twinkle Star
    F4,F4,E4,E4,D4,D4,C4,
    G4,G4,F4,F4,E4,E4,D4,
    G4,G4,F4,F4,E4,E4,D4,
    C4,C4,G4,G4,A4,A4,G4,
    F4,F4,E4,E4,D4,D4,C4
};

uchar code Dur[] = { // 每個音符的播放長度，配合Timer1使用
    2,2,2,2,2,2,2,2,2,
    2,2,2,2,2,2,2,2,
    2,2,2,2,2,2,2,
    2,2,2,2,2,2,
    2,2,2,2,2,2,

    2,2,2,2,2,2,2,2,
    2,2,2,2,3,1,3,
    2,2,2,2,2,2,2,2,
    2,2,2,2,3,1,4,
    2,2,2,2,2,2,

    2,2,2,2,2,2,3,
    2,2,2,2,2,2,3,
    2,2,2,2,2,2,3,
    2,2,2,2,2,2,3,
    2,2,2,2,2,2,3,
    2,2,2,2,2,2,4
};
```

```
};
```

七、系統狀態與動畫控制店數區

```
uchar idx = 0;           // 紀錄目前播放到第幾個音符
uchar tick = 0;          // Timer1用來計算節奏的計數器
bit shortRest = 0;       // 判斷目前是否在音符間的短休止

char lcdText[24] = " Merry Christmas "; // LCD跑馬燈顯示的字串
uchar lcdPos = 0;        // LCD目前顯示的起始位置
uchar lcdTick = 0;       // LCD捲動用的計數器

bit blinkState = 1;      // 控制8*8點陣目前是亮還是暗
uchar blinkTick = 0;     // 控制點陣閃爍速度的計數器

bit smileToggle = 0;     // 控制新年模式下表情是否切換
uchar smileTick = 0;     // 控制表情切換速度的計數器
```

八、延遲函式區

```
void Delay1ms(){
    uint i;               // 宣告一個暫存變數
    for(i=0;i<120;i++);   // 用空迴圈產生約1ms的延遲
}

void delay40us(){
    uchar i;              // 宣告暫存變數
    for(i=0;i<20;i++);    // 產生LCD所需的短延遲
}
```

九、LCD底層控制函式區

```
void Command(uchar cmd){
    P0 = cmd;             // 將指令資料放到P0
    RS = 0;               // 設定為指令模式
    RW = 0;               // 設定為寫入
    EN = 1;               // 拉高EN，準備送出指令
    delay40us();           // 等待LCD接收
    EN = 0;               // 拉低EN，完成一次指令傳送
}

void Data(uchar dat){
    P0 = dat;             // 將要顯示的資料放到P0
    RS = 1;               // 設定為資料模式
    RW = 0;               // 設定為寫入
    EN = 1;               // 拉高EN
    delay40us();           // 等待LCD接收
    EN = 0;               // 拉低EN，完成資料寫入
}

void LCD_Init(){
    Command(0x38);        // 設定LCD為8-bit、2行顯示
    Command(0x0F);        // 開啟顯示、游標與游標閃爍
    Command(0x06);        // 每寫入一個字游標向右移
    Command(0x01);        // 清除LCD畫面
}

void LCD_Scroll(){
    uchar i;              // 宣告計數變數
    Command(0x80);        // 將游標設定到第一行最左邊
    for(i=0;i<16;i++){    // LCD一行顯示16個字
        Data(lcdText[(lcdPos+i)%strlen(lcdText)]); // 循環顯示字串內容
    }
    lcdPos = (lcdPos + 1) % strlen(lcdText); // 下一次從下一個字開始顯示
}
```

十、模式切換與按鍵處理

```
void switchMode(){
    currentMode = (currentMode + 1) % 3; // 模式在三種之間循環切換
    lcdPos = 0;             // 重置LCD顯示位置
}
```

```

tick = 0;           // 重置節奏計數
shortRest = 0;      // 取消短休止狀態
smileTick = 0;      // 重置表情切換計數
smileToggle = 0;    // 表情回到初始狀態

if(currentMode == MODE_XMAS){
    idx = 0;         // 指向聖誕歌曲開頭
    strcpy(lcdText, " Merry Christmas ");
}else if(currentMode == MODE_NEWYR){
    idx = 36;         // 指向快樂頌歌曲開頭
    strcpy(lcdText, " Happy New Year ");
}else{
    idx = 71;         // 指向星星歌曲開頭
    strcpy(lcdText, " Twinkle Star ");
}

TH0 = Song[idx] >> 8; // 設定Timer0高位，準備播放新音符
TL0 = Song[idx];      // 設定Timer0低位
}

void checkButton(){
    static bit last = 1; // 紀錄上一次按鍵狀態
    if(S1 == 0 && last == 1){ // 偵測按鍵由沒按到按下
        Delay1ms();         // 防彈跳延遲
        Delay1ms();
        if(S1 == 0){         // 再確認一次真的有按
            switchMode();    // 執行模式切換
        }
    }
    last = S1;               // 更新按鍵狀態
}

```

十一、Timer0中斷服務程式(音高產生)

```

void Timer0_ISR(void) interrupt 1{
    if(Song[idx] == 0 || shortRest){ // 如果是休止或短休止
        BZ = 0;           // 關閉蜂鳴器
    }else{
        BZ = ~BZ;         // 翻轉蜂鳴器腳位產生方波
    }
    TH0 = Song[idx] >> 8; // 重新載入Timer0高位
    TL0 = Song[idx];      // 重新載入Timer0低位
}

```

十二、Timer1中斷服務程式(節奏與畫面呈現)

```

void Timer1_ISR(void) interrupt 3{
    tick++;               // 節奏計數加一
    lcdTick++;            // LCD捲動計數加一

    blinkTick++;          // 點陣閃爍計數加一
    if(blinkTick >= 6){
        blinkTick = 0;    // 重置閃爍計數
        blinkState = !blinkState; // 切換亮與暗
    }

    if(currentMode == MODE_NEWYR){
        smileTick++;       // 表情切換計數加一
        if(smileTick >= 12){
            smileTick = 0; // 重置表情計數
            smileToggle = !smileToggle; // 切換表情
        }
    }

    if(lcdTick >= 10){
        lcdTick = 0;       // 重置LCD計數
        LCD_Scroll();      // 執行跑馬燈
    }

    if(!shortRest && tick >= Dur[idx] * 2){
        shortRest = 1;     // 進入短休止
    }
}

```

```

    tick = 0;
    return;
}

if(shortRest && tick >= 1){
    shortRest = 0;    // 結束短休止
    tick = 0;
    idx++;            // 前往下一個音符

    if(currentMode == MODE_XMAS && idx > 35) idx = 0;
    if(currentMode == MODE_NEWYR && idx > 70) idx = 36;
    if(currentMode == MODE_STAR && idx > 112) idx = 71;

    TH0 = Song[idx] >> 8; // 設定新音符音高
    TL0 = Song[idx];
}
}

```

十三、8*8 LED點陣顯示

```

void displayPattern(){
    uchar r,p;    // r 表示目前列，p 表示該列資料
    for(r=0;r<8;r++){
        P3 = ~(1<<r);    // 選擇目前要顯示的那一列
        if(blinkState == 0 && currentMode != MODE_NEWYR){
            P2 = 0xFF;    // 關閉整列，製造閃爍效果
        }else{
            if(currentMode == MODE_XMAS)
                p = christmasTree[r]; // 顯示聖誕樹
            else if(currentMode == MODE_NEWYR)
                p = smileToggle ? smileyFace[r] : neutralFace[r]; // 顯示表情
            else
                p = star[r]; // 顯示星星
            P2 = ~p;    // 將資料輸出到點陣
        }
        Delay1ms();    // 短暫停留讓眼睛看到
        P3 = 0xFF;    // 關閉所有列避免殘影
    }
}

```

十四、主程式區(系統初始化與主迴圈)

```

void main(){
    P0 = P1 = P2 = P3 = 0xFF; // 將所有I/O腳位設為高電位

    LCD_Init();    // 初始化LCD

    TMOD = 0x11;    // 設定Timer0、Timer1為Mode 1

    TH0 = Song[0] >> 8;    // 設定Timer0初始音高
    TL0 = Song[0];

    TH1 = 0x3C;    // 設定Timer1重載值
    TL1 = 0xB0;

    ET0 = 1;    // 啟用Timer0中斷
    ET1 = 1;    // 啟用Timer1中斷
    EA = 1;    // 啟用全域中斷

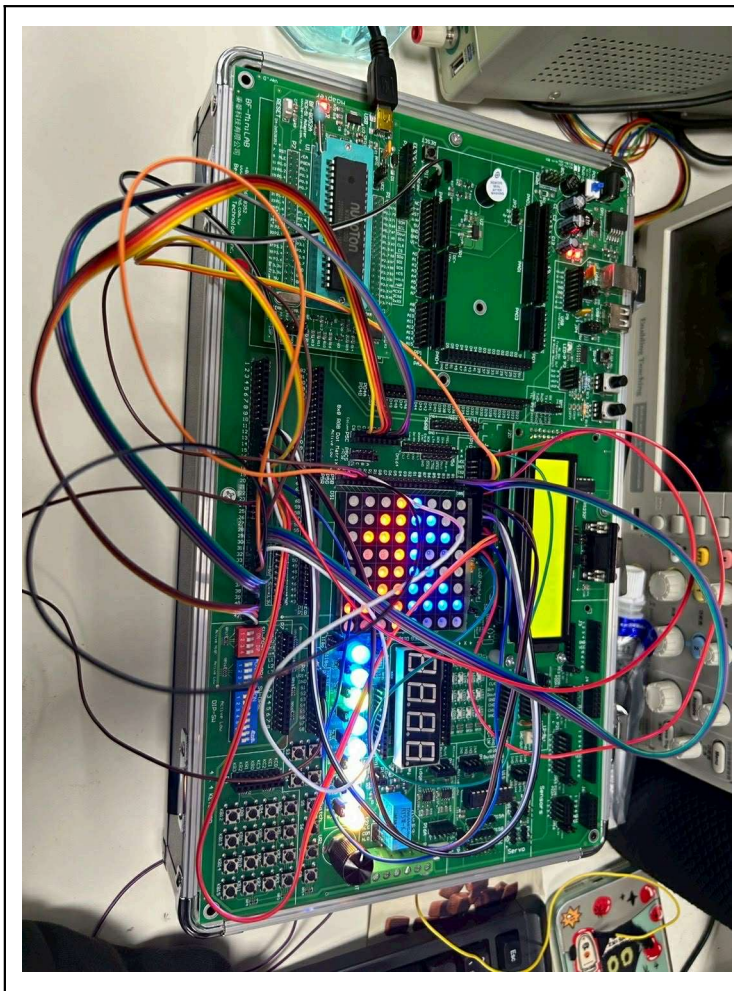
    TR0 = 1;    // 啟動Timer0
    TR1 = 1;    // 啟動Timer1

    while(1){
        displayPattern();    // 持續掃描顯示8*8點陣
        checkButton();    // 持續偵測按鍵切換模式
    }
}

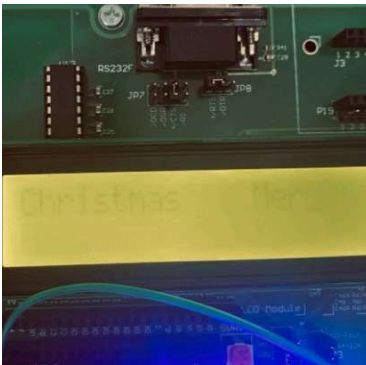
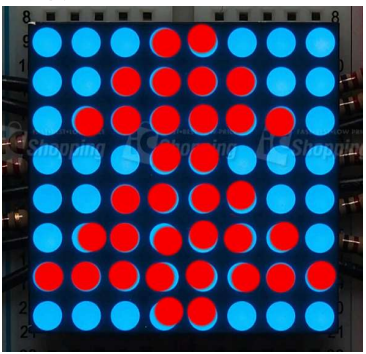
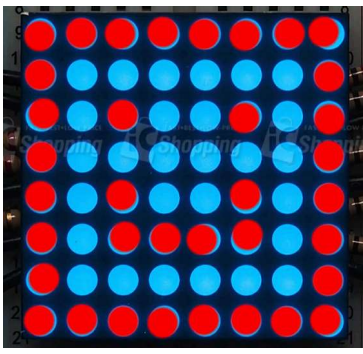
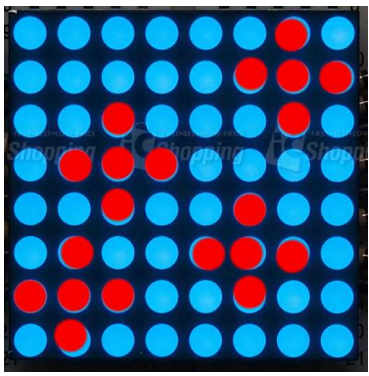
```

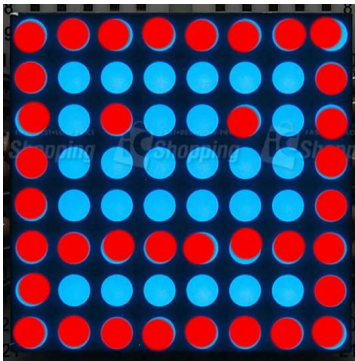
結果實驗照片：

下方為整體插線效果



整體對應結果畫面以及資訊:

模式	聖誕節	快樂頌	小星星
LCD文字列印內容	Merry Christmas	Happy New Year	Twinkle Star
LCD實際畫面 (文字有點淺)			
8*8點陣設計	聖誕樹 	笑  不笑	星星星 

			
8*8點陣畫面	聖誕樹	笑 不笑	星星
音樂	Merry Christmas(最後一句是 Happy New Year)	快樂頌	小星星

問題與心得:

這次期末實驗的過程中，我們其實遇到了不少問題，以下是遇到的:

1. 是在燒錄程式時，一直出現chips not ready的錯誤訊息，不管是更換板子、電腦，甚至連電源線都換過，燒錄軟體仍然不斷要求我們對硬體進行reset，一開始完全找不到原因，後來經過反覆測試與檢查程式後，才發現問題並不是出在燒錄設備，而是程式本身的設計方式，因為在8051微控制器中我們同時使用了Timer中斷(ISR)以及以delay為基礎的I/O的掃描方式，而且讓中斷服務程式與主程式在不同時間點同時操作同一組I/O Port(P0、P1、P2)，導致8*8 LED與LCD的控制互相衝突，由於8051微控制器本身沒有提供能確保I/O操作一次完成、不會被中斷打斷的機制，也缺乏硬體層級的互斥設計，在中斷巢狀發生時，很容易讓I/O匯流排進入不穩定或錯誤狀態，進而造成晶片非預期重置或系統當機；另外，我們一開始在中斷服務程式中宣告區域變數並寫了較複雜的流程控制，也讓8051本來就不大的RAM stack更容易發生溢位(Stack爆掉)，使系統更加不穩定，後來我們調整設計方式，改成8*8 LED仍由interrupt控制，而LCD的EN脈波改用delay的方式產生，將兩者分開處理後，燒錄與系統不穩定的問題就順利解決了。
2. 是LCD無法正常顯示文字，雖然背光有亮，但畫面上完全沒有字，為了解決這個問題，我們回去參考之前LAB 的範例程式，重新檢查LCD的接線與程式配置方式，後來發現原本將LCD的DB0~DB7全部接到P0.0~P0.7的做法會導致顯示異常(甚至試燒入失敗)，因此我們改成依照之前LAB成功顯示的接線與寫法為主，再重新整合後續的蜂鳴器與8*8 LED，LCD顯示的問題才得以解決。
3. 最後還遇到一個相對較小的問題，就是8*8 LED顯示的圖案位置不正確。一開始直接用十六進位設定圖案時比較不容易看出錯誤，後來我們先用 8-bit 的方式(0 表示不亮、1 表示亮)把每一列的點陣寫清楚，再轉換回十六進位修改程式，確認每一個點的位置後，圖案顯示就方便觀察、更改，也正確了。